

นายธนินทร อินทรบุตร 673380043-6 Sec.2

ENDPOINT ทั้งหมดของ Lab10

GET ID <http://localhost:8080/products/1>

The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:8080/products/1`
- Method:** `GET`
- Response Status:** `200 OK`, `6 ms`, `299 B`
- Response Body (JSON):**

```
1 {
2   "id": "1",
3   "name": "iPhone 15 Pro (673380043-6 นายธนินทร อินทรบุตร SEC 1)",
4   "category": "Electronics",
5   "brand": "Apple",
6   "stock": 50,
7   "price": 39900.0,
8   "discountType": "MEMBER",
9   "discountedPrice": 35910.0
10 }
```

GET <http://localhost:8080/products>

http://localhost:8080/products

GET http://localhost:8080/products

Params Authorization Headers (6) Body Pre-request Script Tests Settings

Query Params

Key	Value
Key	Value

Body Cookies Headers (2) Test Results

Status: 200 OK Time: 19 ms Size: 620 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "id": "1",
3   "name": "iPhone 15 Pro (67339943-6 uneSuns #rompes SEC 1)",
4   "category": "Electronics",
5   "brand": "Apple",
6   "stock": 58,
7   "price": 39900.0,
8   "discountType": "MEMBER",
9   "discountedPrice": 38910.0
10 }
11 {
12   "id": "2",
13   "name": "MacBook Air M3",
14   "category": "Electronics",
15   "brand": "Apple",
16   "stock": 26,
17   "price": 49900.0,
18   "discountType": "NONE",
19   "discountedPrice": 49900.0
20 }
21 {
22   "id": "3",
23   "name": "Samsung Galaxy S24",
24   "category": "Electronics",
25   "brand": "Samsung",
26   "stock": 38,
27   "price": 29900.0,
28   "discountType": "SEASONAL",
29 }
```

POST <http://localhost:8080/products>

http://localhost:8080/products

POST http://localhost:8080/products

Params Authorization Headers (8) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary JSON

raw

```
1 {
2   "name": "iPad Pro",
3   "category": "Electronics",
4   "brand": "Apple",
5   "stock": 10,
6   "price": 32900.0,
7   "userType": "MEMBER"
8 }
```

Body Cookies Headers (2) Test Results

Pretty Raw Preview Visualize JSON

```
1 {
2   "id": "2876aecb-066f-4258-b217-256c81f83c16",
3   "name": "iPad Pro",
4   "category": "Electronics",
5   "brand": "Apple",
6   "stock": 10,
7   "price": 32900.0,
8   "discountType": null,
9   "discountedPrice": 32900.0
10 }
```

DELETE http://localhost:8080/products/3

The screenshot shows a REST client interface with the following components:

- URL Bar:** Displays the URL `http://localhost:8080/products/3`.
- Method:** A dropdown menu set to `DELETE`.
- Tabs:** Includes `Params`, `Authorization`, `Headers (8)`, `Body` (selected), `Pre-request Script`, `Tests`, and `Settings`.
- Body Type:** Radio buttons for `none`, `form-data`, `x-www-form-urlencoded`, `raw`, and `binary`. A `JSON` dropdown is also present.
- Body Editor:** A text area containing a JSON object:

```
1  {
2    "name": "iPad Pro",
3    "category": "Electronics",
4    "brand": "Apple",
5    "stock": 10,
6    "price": 32900.0,
7    "userType": "MEMBER"
8  }
```
- Response Section:** Includes tabs for `Body`, `Cookies`, `Headers (1)`, and `Test Results`. Below these are buttons for `Pretty`, `Raw`, `Preview`, and `Visualize`. A `Text` dropdown and a copy icon are also visible.
- Response Content:** The number `1` is displayed below the response section.

GET http://localhost:8080/products/category/Electronics

http://localhost:8080/products/category/Electronics

GET http://localhost:8080/products/category/Electronics

Params Authorization Headers (6) **Body** Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary

This request does not have a

Body Cookies Headers (2) Test Results

Pretty Raw Preview Visualize JSON

```
1 {
2   {
3     "id": "1",
4     "name": "iPhone 15 Pro (673380043-6 นานาพันธุ์, สิบพันต่อ SEC 1)",
5     "category": "Electronics",
6     "brand": "Apple",
7     "stock": 50,
8     "price": 39900.0,
9     "discountType": "MEMBER",
10    "discountedPrice": 35910.0
11  },
12  {
13    "id": "2",
14    "name": "MacBook Air M3",
15    "category": "Electronics",
16    "brand": "Apple",
17    "stock": 20,
18    "price": 49900.0,
19    "discountType": "NONE",
20    "discountedPrice": 49900.0
21  },
22  {
23    "id": "2876aecb-866f-4258-b217-256c81f83c16",
24    "name": "iPad Pro",
25    "category": "Electronics",
26    "brand": "Apple",
27    "stock": 10,
28    "price": 32900.0,
29    "discountType": null,
30    "discountedPrice": 32900.0
31  }
32 }
```

GET http://localhost:8080/products/1/price

GET http://localhost:8080/products/1/price

Params Authorization Headers (6) **Body** Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary

Body Cookies Headers (2) Test Results

Pretty Raw Preview Visualize JSON

```
1 35910.0
```

PDF ต้องอธิบาย:

Reactive Programming vs Blocking — ต่างกันอย่างไร

Blocking I/O (Traditional Spring MVC): ใช้รูปแบบ **Thread-per-Request** เมื่อมี Request เข้ามา เซิร์ฟเวอร์จะจอง 1 Thread เพื่อประมวลผล หากต้องรอดึงข้อมูลจาก Database หรือ External API ตัว Thread นั้นจะถูกบล็อก (Block) อยู่ในสถานะรอคอย ทำให้สิ้นเปลืองทรัพยากร และรองรับ Concurrent Users จำนวนมากพร้อมกันได้ยาก

Reactive Programming (Spring WebFlux): ใช้รูปแบบ **Non-blocking Event Loop (Asynchronous)** โดยใช้ Thread จำนวนน้อย (Event Loop Threads) ในการรับ Request และส่งต่องานในรูปแบบ Event/Publisher เมื่อมี Task I/O เกิดขึ้น Thread จะไม่นั่งรอ แต่จะไปทำงานอื่นต่อ และเมื่อข้อมูลพร้อมแล้วระบบจะแจ้งเตือนกลับมา (Callback / Reactive Stream) ทำให้ระบบประหยัดทรัพยากร memory และรองรับ High Concurrency ได้อย่างมีประสิทธิภาพ

Mono vs Flux — ใช้กรณีไหน และทำไม

ทั้งคู่เป็นคลาสพื้นฐานตามมาตรฐาน Reactive Streams (Publisher) ของ Project Reactor:

- **Mono<T>:**
 - **คืออะไร:** ปล่อยข้อมูลออกมา 0 หรือ 1 Element แล้วส่งสัญญาณ onComplete หรือ onError
 - **กรณีใช้งาน:** เหมาะสำหรับผลลัพธ์ที่เป็นข้อมูลรายการเดียว เช่น การค้นหาตาม ID (findById), การบันทึกข้อมูล (save), การส่งสถานะความสำเร็จว่างเปล่า (Mono<Void>) หรือค่าการคำนวณเดี่ยวๆ
- **Flux<T>:**
 - **คืออะไร:** ปล่อยข้อมูลออกมาเป็นสายธาร (Stream) จำนวน 0 ถึง N Elements (ไม่จำกัด)
 - **กรณีใช้งาน:** เหมาะสำหรับข้อมูลแบบชุด/คอลเลกชัน เช่น การดึงรายการสินค้าทั้งหมด (findAll), การกรองข้อมูลตามประเภท (findByCategory) หรือข้อมูลที่ส่งมาแบบ Streaming Real-time

WebClient — อธิบาย method chain `.get()`, `.uri()`, `.retrieve()`, `.bodyToMono/Flux()`

WebClient เป็น Non-blocking HTTP Client สำหรับส่ง Request ไปยัง External API มีกระบวนการเรียกใช้งานแบบ Method Chaining ดังนี้:

1. `.get()` / `.post()` / `.delete()`: กำหนด HTTP Method ที่ต้องการส่งไปยังปลายทาง
2. `.uri("/products/{id}", id)`: กำหนด Endpoint URL พร้อมรองรับการแทนค่า URI Variables
3. `.retrieve()`: สั่งให้เริ่มทำ HTTP Request และดึง Response Body กลับมา
4. `.bodyToMono(Product.class)` / `.bodyToFlux(Product.class)`: แปลง Response Body ที่ได้จาก HTTP Response ให้เป็น `Mono<T>` (กรณีรับข้อมูล 1 ชิ้น) หรือ `Flux<T>` (กรณีรับข้อมูลเป็น Array/Stream) ตามลำดับ

Code ของ Repository, Service, Controller พร้อม comment อธิบาย

โค้ดที่แก้ไขแล้ว ของ Repository

```
package com.example.lab10.repository;

import com.example.lab10.model.Product;
import reactor.core.publisher.Flux;
import reactor.core.publisher.Mono;

import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

/**
 * ProductRepository — In-memory Reactive Repository
 */
public class ProductRepository {
```

// — In-memory storage —

```
private final Map<String, Product> store = new ConcurrentHashMap<>();
```

// — Constructor: ใส่ข้อมูลตัวอย่าง —

```
public ProductRepository() {  
  
    store.put("1", new Product("1", "iPhone 15 Pro (67XXXXXXXX-X SEC 1)",  
                                "Electronics", "Apple", 50, 39900.0, "MEMBER"));  
  
    store.put("2", new Product("2", "MacBook Air M3",  
                                "Electronics", "Apple", 20, 49900.0, "NONE"));  
  
    store.put("3", new Product("3", "Samsung Galaxy S24",  
                                "Electronics", "Samsung", 30, 29900.0, "SEASONAL"));  
  
}
```

// — 1. หา Product 1 รายการ —

```
public Mono<Product> findById(String id) {  
  
    Product product = store.get(id);  
  
    return Mono.justOrEmpty(product);  
  
}
```

// — 2. หา Product ทั้งหมด —

```
public Flux<Product> findAll() {  
  
    return Flux.fromIterable(store.values());  
  
}
```

// — 3. บันทึก Product —

```
public Mono<Product> save(Product product) {
```

```
store.put(product.getId(), product);

return Mono.just(product);
}
```

```
// — 4. ลบ Product —————
```

```
public Mono<Void> deleteById(String id) {

    store.remove(id);

    return Mono.empty();

}
```

```
// — 5. กรองตาม category —————
```

```
public Flux<Product> findByCategory(String category) {

    return findAll()

        .filter(p -> p.getCategory().equalsIgnoreCase(category));

}

}
```

โค้ดที่แก้ไขแล้วของ Service

```
package com.example.lab10.service;

import com.example.lab10.model.Product;
import com.example.lab10.repository.ProductRepository;
import org.springframework.stereotype.Service;
import reactor.core.publisher.Flux;
import reactor.core.publisher.Mono;

import java.util.UUID;
```

```

/**
 * ProductService — Business Logic Layer
 */
@Service
public class ProductService {

    // — Constructor Injection (DIP — SOLID) —
    private final ProductRepository repository;

    public ProductService(ProductRepository repository) {
        this.repository = repository;
    }

    // — 1. ดึง Product 1 รายการ —
    public Mono<Product> getByid(String id) {
        return repository.findById(id)
            .switchIfEmpty(Mono.error(new RuntimeException("Product not found: " + id)));
    }

    // — 2. ดึง Product ทั้งหมด —
    public Flux<Product> getAll() {
        return repository.findAll();
    }

    // — 3. บันทึก Product —
    public Mono<Product> save(Product product) {
        if (product.getId() == null || product.getId().trim().isEmpty()) {

```

```

        product.setId(UUID.randomUUID().toString());
    }

    return repository.save(product);
}

```

// — 4. ลบ Product —————

```

public Mono<Void> delete(String id) {

    return repository.deleteById(id);

}

```

// — 5. กรองตาม category —————

```

public Flux<Product> getByCategory(String category) {

    return repository.findByCategory(category);

}

```

// — 6. คำนวณราคาหลังส่วนลด —————

```

public Mono<Double> getDiscountedPrice(String id) {

    return getById(id)

        .map(Product::getDiscountedPrice);

}
}

```

โค้ดที่แก้ไขแล้วของ Controller

```

package com.example.lab10.controller;

import com.example.lab10.model.Product;
import com.example.lab10.service.ProductService;
import org.springframework.web.bind.annotation.*;

```

```
import reactor.core.publisher.Flux;

import reactor.core.publisher.Mono;

/**
 * ProductController — Reactive REST Controller
 */

@RestController

@RequestMapping("/products")

public class ProductController {

    // — Constructor Injection (DIP — SOLID) —

    private final ProductService service;

    public ProductController(ProductService service) {

        this.service = service;

    }

    // =====

    //  ตัวอย่างที่ทำเสร็จแล้ว

    // =====

    /**
     * GET /products/{id}
     *
     * คืน Mono<Product> — ค้นหา Product 1 รายการ
     */

    @GetMapping("/{id}")

    public Mono<Product> getById(@PathVariable String id) {
```

```
        return service.getByld(id);  
    }  
}
```

```
// =====  
// ❌ Endpoints ที่เดิมเพิ่มจนครบสมบูรณ์  
// =====
```

```
/**  
 * GET /products  
 * คืน Flux<Product> ทุกรายการ  
 */
```

```
@GetMapping  
public Flux<Product> getAll() {  
    return service.getAll();  
}
```

```
/**  
 * POST /products  
 * รับ Product จาก request body แล้วบันทึก  
 */
```

```
@PostMapping  
public Mono<Product> save(@RequestBody Product product) {  
    return service.save(product);  
}
```

```
/**  
 * DELETE /products/{id}
```

```

* ลบ Product และคืน Mono<Void>

*/

@DeleteMapping("/{id}")

public Mono<Void> delete(@PathVariable String id) {

    return service.delete(id);

}

/**

* GET /products/category/{category}

* คืน Flux<Product> ที่กรองตาม category

*/

@GetMapping("/category/{category}")

public Flux<Product> getByCategory(@PathVariable String category) {

    return service.getByCategory(category);

}

/**

* GET /products/{id}/price

* คืน Mono<Double> ราคาหลังส่วนลด

*/

@GetMapping("/{id}/price")

public Mono<Double> getDiscountedPrice(@PathVariable String id) {

    return service.getDiscountedPrice(id);

}

}

```

โค้ดที่แก้ไขแล้วของ WebClient

```
package com.example.lab10.client;
```

```
import com.example.lab10.model.Product;

import org.springframework.stereotype.Component;

import org.springframework.web.reactive.function.client.WebClient;

import reactor.core.publisher.Flux;

import reactor.core.publisher.Mono;
```

```
/**
```

```
 * ProductWebClient — Reactive HTTP Client
```

```
 */
```

```
@Component
```

```
public class ProductWebClient {
```

```
    //  WebClient ตั้งค่าแล้ว ขึ้นที่ server ตัวเอง
```

```
    private final WebClient client = WebClient.create("http://localhost:8080");
```

```
    // =====
```

```
    //  ตัวอย่างที่ทำเสร็จแล้ว
```

```
    // =====
```

```
/**
```

```
 * GET /products/{id} → Mono<Product>
```

```
 * ดึง Product 1 รายการจาก server
```

```
 */
```

```
public Mono<Product> getProductById(String id) {
```

```
    return client.get()
```

```
        .uri("/products/{id}", id)
```

```
        .retrieve()

        .bodyToMono(Product.class);
    }
}
```

```
// =====
```

```
// ❌ Endpoints ที่เดิมเพิ่มจนครบสมบูรณ์
```

```
// =====
```

```
/**
```

```
 * GET /products → Flux<Product>
```

```
 * ดึง Product ทั้งหมดจาก server
```

```
 */
```

```
public Flux<Product> getAllProducts() {
```

```
    return client.get()
```

```
        .uri("/products")
```

```
        .retrieve()
```

```
        .bodyToFlux(Product.class);
```

```
}
```

```
/**
```

```
 * POST /products → Mono<Product>
```

```
 * ส่ง Product ใหม่ไปยัง server
```

```
 */
```

```
public Mono<Product> createProduct(Product product) {
```

```
    return client.post()
```

```
        .uri("/products")
```

```
        .bodyValue(product)
```

```
        .retrieve()

        .bodyToMono(Product.class);
    }
}
```

```
/**
```

```
 * DELETE /products/{id} → Mono<Void>
```

```
 * ส่ง request ลบ Product
```

```
 */
```

```
public Mono<Void> deleteProduct(String id) {
```

```
    return client.delete()
```

```
        .uri("/products/{id}", id)
```

```
        .retrieve()
```

```
        .bodyToMono(Void.class);
```

```
}
```

```
/**
```

```
 * GET /products/category/{category} → Flux<Product>
```

```
 * ดึง Product ตาม category
```

```
 */
```

```
public Flux<Product> getByCategory(String category) {
```

```
    return client.get()
```

```
        .uri("/products/category/{category}", category)
```

```
        .retrieve()
```

```
        .bodyToFlux(Product.class);
```

```
}
```

```
/**
```

* GET /products/{id}/price → Mono<Double>

* ดึงราคาหลังส่วนลด แล้ว log ค่าลง console

*/

```
public Mono<Double> getDiscountedPrice(String id) {  
  
    return client.get()  
  
        .uri("/products/{id}/price", id)  
  
        .retrieve()  
  
        .bodyToMono(Double.class)  
  
        .doOnNext(price -> System.out.println("Price: " + price));  
  
    }  
}
```